

DPDK下CPU与内存规划

- [dpdk资源规划](#)
 - [内存资源规划](#)
 - [CPU资源规划](#)
 - [网卡资源规划](#)
- [611资源规划](#)

dpdk资源规划 [🔗](#)

dpdk对接包要求所有业务网卡需要在一个numa node上，否则报错

内存资源规划 [🔗](#)

	架构	全部巨页内存GiB	PMD巨页内存GiB	计算巨页内存GiB
网络节点、裸金属网关节点、二层桥接节点	all	最大32G per numa node，所以hugepage最大32*numa nodes，剩余的非hugepage内存给系统和pods 否则按下面公式计算，结果除以node nodes，获得每个numa node的huge page内存 $hugepages = total - sys:4 - pods:40$	最大32G per numa 非pmd线程使用的非大页内存亲和网卡所在socket	-
计算节点	all	$hugepages = total - sys:32 - pods:10$ 如果是多numa架构，会保证一个numa至少有8G的非大页，所以最终的非大页内存预留=MAX (nodesx8，42)	per numa: 4 非pmd线程使用的非大页内存亲和网卡所在socket	$hugepages - 4 * numa\ nodes$
计算存储节点	all	$hugepages = total - osd:nx4 - sys:32 - pods:10$	per numa: 4 非pmd线程使用的非大页内存亲和网卡所在socket	$hugepages - 4 * numa\ nodes$
控制计算存储				
控制				
控制存储 (生产不支持)				

说明

- 内存资源存储节点每个osd预留4g的内存；
- pmd使用的巨页内存，剩余的都留给kvm使用；

CPU资源规划 [🔗](#)

	架构	预留CPU	PMD CPU	计算CPU
网络节点、裸金属网关节点、二层桥接节点	x86_s2n8(hygon)	-	使用网卡所在cpu socket的一半cpu，剩余一半给非pmd线程使用	-
	代表2个socket，每个socket 4个 numa node			
	arm_s2n4(kunpeng 920)	-	同上	-
	arm_s2n16(s2500)	-	同上	
	其他cpu	-	同上	
计算节点	x86/hygon	-	numa_pmd=-4: -4: 代表使用numa node里后4个cpu，比如 node0 CPU(s): 0-7，则使用4-7，比如 node0 CPU(s): 0-7,16-23，则使用6-7,22-23，以下类似 计算节点只使用业务网卡对应numa node的 4个cpu 非pmd线程亲和网卡所在cpu socket的前两个numa	vm cpu - numa_pmd
计算存储节点	x86/hygon	-	numa_pmd=-4: 非pmd线程亲和网卡所在cpu socket的前两个numa	vm cpu - numa_pmd
控制计算存储控制				
控制存储 (生产不支持)				

说明

- 业务网卡要求插入在一个numa node上，或者一个socket上，但是需要保证一个socket上的numa node是确定的，比如2个网卡插入到socket 0上，socket 0 有4个numa node，则2个网卡的numa node应该是一样的
- numa_pmd占用系统预留影响不大，原来业务网络处理也是系统预留的cpu处理，只不过限制业务网络处理是独享CPU。
- 在纯网络节点numa_pmd的占用不会影响系统cpu，通常我们将业务网络的网卡都插入同一个socket上面，也就意味着另外一个socket上面的所有cpu都可以给系统使用。

网卡资源规划

	架构	管理网	生产网	外部网络	裸金属L2网络	存储网
网络	x86/arm	bond0:非dpdk	bond1:dpdk	bond1:dpdk	bond2:dpdk	-
计算	x86/arm	bond0:非dpdk	bond1:dpdk	-	-	bond2:非dpdk
计算存储	x86/arm	bond0:非dpdk	bond1:dpdk	-	-	bond2:非dpdk

控制计算存储						
控制						
控制存储						
(生产不支持)						

说明

- 计算或计算存储，生产网络的bond需要独享，不能与其他网络角色混用
- 生产网、外部网络、裸金属网络均开启dpdk支持；
- 网络节点的生产网，外部网络，裸金属网络可以在一起，也可以完全分开

611资源规划

v611版本资源划分方案:

v611	memory	cpuset
控制+计算+存储	pods: 40G vms: $\text{total_memory} - \text{pods}(40\text{G}) - \text{system}(3\text{G}) - \text{ceph}(4\text{G} * N)$	x86_pods: 8 threads(NUMA0 first 8 threads) arm_pods: 16 threads(first 8*2 threads) s2500_pods: 不限制 s2500_ceph: 16-63 x86_vms: $\text{total_cpuset} - \text{pods}(8 \text{ threads}) - \text{system}(2 \text{ threads})$ arm_vms: $\text{total_cpuset} - \text{pods}(8 * 2 \text{ threads}) - \text{system}(2 \text{ threads})$
存储+计算	pods: 10G vms: $\text{total_memory} - \text{pods}(10\text{G}) - \text{system}(3\text{G}) - \text{ceph}(4\text{G} * N)$	kubepods: 不限制 s2500_ceph: 16-63 x86_vms: $\text{total_cpuset} - \text{pods}(2 \text{ threads}) - \text{system}(2 \text{ threads})$ arm_vms: $\text{total_cpuset} - \text{pods}(2 * 2 \text{ threads}) - \text{system}(2 \text{ threads})$
计算	pods: 10G vms: $\text{total_memory} - \text{pods}(10\text{G}) - \text{system}(3\text{G})$	kubepods: 不限制 x86_vms: $\text{total_cpuset} - \text{pods}(2 \text{ threads}) - \text{system}(2 \text{ threads})$ arm_vms: $\text{total_cpuset} - \text{pods}(2 * 2 \text{ threads}) - \text{system}(2 \text{ threads})$
存储	pods: 10G	不限制 s2500_ceph: 16-63
控制	pods: $\text{total_memory} - 20\text{G}$	不限制
控制+存储	pods: $\text{total_memory} - 20\text{G}$	不限制 s2500_ceph: 16-63
安全容器	pods: $\text{total_memory} - 10\text{G}$	不限制
云产品节点	虚拟机节点: $\text{pods: total_memory} - \text{system}(3\text{G})$ 物理节点: $\text{pods: total_memory} - \text{system}(10\text{G})$	不限制

https://dev.easystack.cn/rd/review/64657/RD/components/5.0.1/certified_architecture/roles_separation_deployment.html